Titotto Paolo 895227

# REPORT CHALLENGE 1

## Sommario

REPORT CHALLENGE 1 ..... 1
POSIZIONAMENTO NELLA DIRECOTY E AVVIAMENTO GDB PASSWORD ..... 2
SET DISASSEMBLY INTEL E DISASS MAIN CON BREAKPOINT ..... 2
CREAZIONE BREAKPOINT ALL'ISTRUZIONE DI INTERESSE ..... 4
DEBUGGIN CODICE ..... 4
ESECUZIONE DEL PROGRAMMA E CONTROLLO PASSWORD ..... 7

Titotto Paolo 895227

## POSIZIONAMENTO NELLA DIRECOTY E AVVIAMENTO GDB PASSWORD

Come prima azione mi sono spostato nella directory Challenge1 e ho avviato il debug su password, come mostrato in figura:

```
rookie@8d3caaeb9e26:~/Challenge1$ gdb password
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from password...(no debugging symbols found)...done.
```

## SET DISASSEMBLY INTEL E DISASS MAIN CON BREAKPOINT

Successivamente ho eseguito set disassmbly intel in modo da disassemblare il codice macchina con la sintassi intel. Figura 2:

```
(gdb) set disassembly intel
```

Come prossima istruzione mi creo un breakpoint al main, quindi all'inizio del programma in modo da avere un punto di riferimento quando andrò a eseguirlo con il debug. La figura sottostante mostra il codice scritto nella shell.

```
(gdb) br main
Breakpoint 1 at 0x76e
```

Titotto Paolo 895227

Nelle immagini che seguono si può vedere il main disassemblato in linguaggio assembly tramite l'istruzione disass main. Facendo questo mi è permesso vedere il funzionamento del codice e capire dove verrà fatto il controllo tra la mia stringa che metto come password e quella corretta. Seguo il codice visualizzando le istruzioni assembly con i rispettivi registri.

```
(gdb) disass main
Dump of assembler code for function main:
   0x000000000000076a <+0>:     push   rbp
   0x000000000000076b <+1>:     mov    rbp,rsp
   0x000000000000076e <+4>:     push   rbx
   0x000000000000076f <+5>:     sub    rsp,0x58
   0x0000000000000773 <+9>:     mov    DWORD PTR [rbp-0x54],edi
   0x0000000000000776 <+12>:    mov    QWORD PTR [rbp-0x60],rsi
   0x000000000000077a <+16>:    mov    rax,QWORD PTR fs:0x28
   0x0000000000000783 <+25>:    mov    QWORD PTR [rbp-0x18],rax
   0x0000000000000787 <+29>:    xor    eax,eax
   0x0000000000000789 <+31>:    mov    DWORD PTR [rbp-0x40],0xf
   0x0000000000000790 <+38>:    mov    DWORD PTR [rbp-0x3c],0x16
   0x0000000000000797 <+45>:    mov    DWORD PTR [rbp-0x38],0x3
   0x000000000000079e <+52>:    mov    DWORD PTR [rbp-0x34],0xffffffd0
   0x00000000000007a5 <+59>:    mov    DWORD PTR [rbp-0x30],0x0
   0x00000000000007ac <+66>:    mov    DWORD PTR [rbp-0x2c],0x4
   0x00000000000007b3 <+73>:    mov    DWORD PTR [rbp-0x28],0xf
   0x00000000000007ba <+80>:    mov    DWORD PTR [rbp-0x24],0x15
   0x00000000000007c1 <+87>:    mov    rax,QWORD PTR [rbp-0x60]
   0x00000000000007c5 <+91>:    mov    rax,QWORD PTR [rax+0x8]
   0x00000000000007c9 <+95>:    mov    QWORD PTR [rbp-0x48],rax
   0x00000000000007cd <+99>:    cmp    DWORD PTR [rbp-0x54],0x1
   0x00000000000007d1 <+103>:   je     0x7f3 <main+137>
   0x00000000000007d3 <+105>:   mov    rax,QWORD PTR [rbp-0x48]
   0x00000000000007d7 <+109>:   mov    rdi,rax
   0x00000000000007da <+112>:   call   0x610 <strlen@plt>
   0x00000000000007df <+117>:   mov    rbx,rax
   0x00000000000007e2 <+120>:   lea    rdi,[rip+0x200887]        # 0x201070 <encPassword>
   0x00000000000007e9 <+127>:   call   0x610 <strlen@plt>
   0x00000000000007ee <+132>:   cmp    rbx,rax
   0x00000000000007f1 <+135>:   je     0x827 <main+189>
   0x00000000000007f3 <+137>:   lea    rdi,[rip+0x200876]        # 0x201070 <encPassword>
   0x00000000000007fa <+144>:   call   0x610 <strlen@plt>
   0x00000000000007ff <+149>:   mov    rdx,rax
   0x0000000000000802 <+152>:   mov    rax,QWORD PTR [rbp-0x60]
   0x0000000000000806 <+156>:   mov    rax,QWORD PTR [rax]
   0x0000000000000809 <+159>:   mov    rsi,rax
   0x000000000000080c <+162>:   lea    rdi,[rip+0x125]        # 0x938
   0x0000000000000813 <+169>:   mov    eax,0x0
   0x0000000000000818 <+174>:   call   0x620 <printf@plt>
   0x000000000000081d <+179>:   mov    edi,0x2
```

```
0x0000000000000822 <+184>:   call   0x640 <exit@plt>
0x0000000000000827 <+189>:   mov    DWORD PTR [rbp-0x4c],0x0
0x000000000000082e <+196>:   jmp    0x863 <main+249>
0x0000000000000830 <+198>:   mov    eax,DWORD PTR [rbp-0x4c]
0x0000000000000833 <+201>:   movsxd rdx,eax
0x0000000000000836 <+204>:   mov    rax,QWORD PTR [rbp-0x48]
0x000000000000083a <+208>:   add    rax,rdx
0x000000000000083d <+211>:   movzx  eax,BYTE PTR [rax]
0x0000000000000840 <+214>:   mov    edx,eax
0x0000000000000842 <+216>:   mov    eax,DWORD PTR [rbp-0x4c]
0x0000000000000845 <+219>:   cdqe
0x0000000000000847 <+221>:   mov    eax,DWORD PTR [rbp+rax*4-0x40]
0x000000000000084b <+225>:   lea    ecx,[rdx+rax*1]
0x000000000000084e <+228>:   mov    eax,DWORD PTR [rbp-0x4c]
0x0000000000000851 <+231>:   movsxd rdx,eax
0x0000000000000854 <+234>:   mov    rax,QWORD PTR [rbp-0x48]
0x0000000000000858 <+238>:   add    rax,rdx
0x000000000000085b <+241>:   mov    edx,ecx
0x000000000000085d <+243>:   mov    BYTE PTR [rax],dl
0x000000000000085f <+245>:   add    DWORD PTR [rbp-0x4c],0x1
0x0000000000000863 <+249>:   cmp    DWORD PTR [rbp-0x4c],0x7
0x0000000000000867 <+253>:   jle    0x830 <main+198>
0x0000000000000869 <+255>:   mov    rax,QWORD PTR [rbp-0x48]
0x000000000000086d <+259>:   mov    rsi,rax
0x0000000000000870 <+262>:   lea    rdi,[rip+0x2007f9]        # 0x201070 <encPassword>
0x0000000000000877 <+269>:   call   0x630 <strcmp@plt>
0x000000000000087c <+274>:   test   eax,eax
0x000000000000087e <+276>:   jne    0x896 <main+300>
0x0000000000000880 <+278>:   lea    rdi,[rip+0xd1]        # 0x958
0x0000000000000887 <+285>:   call   0x600 <puts@plt>
0x000000000000088c <+290>:   mov    edi,0x0
0x0000000000000891 <+295>:   call   0x640 <exit@plt>
0x0000000000000896 <+300>:   lea    rdi,[rip+0xe4]        # 0x981
0x000000000000089d <+307>:   call   0x600 <puts@plt>
0x00000000000008a2 <+312>:   mov    edi,0x1
0x00000000000008a7 <+317>:   call   0x640 <exit@plt>
nd of assembler dump.
```

## CREAZIONE BREAKPOINT ALL'ISTRUZIONE DI INTERESSE

A questo punto, dopo aver individuato la cmp corretta e aver capito che il controllo sull'uguaglianza delle stringhe viene fatta dalla strcmp, mi vado a creare un nuovo breakpoint in un indirizzo specifico utilizzando '*' e inserendo in questo caso main+269 che è il punto della strcmp.

```
(gdb) br *main+269
Breakpoint 2 at 0x877
```

## DEBUGGIN CODICE

Infine inizio a runnare il codice con il debugger con il comando run e mettendo una prima password di 8 caratteri e verifico se questa è corretta andando al prossimo breakpoint con 'c' e osservando se il registro della mia password cifrata '$rsi' e quella della password corretta '$rdi' contengono la stessa password. Per trovare la password non cifrata sono partito inserendo una stringa di 8 caratteri e quando ho eseguito ho osservato il risultato cifrato e l'ho confrontato con il risultato atteso, dopodiché ho calcolato la differenza dei valori ASCII tra la lettera che ho inserito e quella risultate dopo la cifratura. Fatto questo ho prima

sottratto il valore al codice ASCII della password corretta e in caso questa differenza risultasse errata provavo a eseguire la somma tra il valore della lettera e il calcolo della lettera cifrata e non. Una volta trovato il fattore ci cifratura procedo a ripetere l'operazione di run cambiando alcune componenti della password per vedere se corrispondono alla password corretta.

```
(gdb) run gaaaaaaa
Starting program: /home/rookie/Challenge1/password gaaaaaaa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vwd1aepv"
(gdb) run g0aaaaaa
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g0aaaaaa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vFd1aepv"
(gdb) run g00aaaaa
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g00aaaaa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF31aepv"
(gdb) run g00daaaa
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g00daaaa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.
```

```
Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF34aepv"
(gdb) run g00djaaa
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g00djaaa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF34jepv"
(gdb) run g00dj0aa
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g00dj0aa

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF34j4pv"
(gdb) run g00dj0ba
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rookie/Challenge1/password g00dj0ba

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF34j4qv"
(gdb) run g00dj0b!
The program being debugged has been started already.
Start it from the beginning? (y or n) y
```

```
Starting program: /home/rookie/Challenge1/password g00dj0b!

Breakpoint 1, 0x000055555540076e in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x0000555555400877 in main ()
(gdb) x/s $rsi
0x7fffffffe8c9: "vF34j4q6"
(gdb) x/s $rdi
0x555555601070 <encPassword>:   "vF34j4q6"
```

## ESECUZIONE DEL PROGRAMMA E CONTROLLO PASSWORD

Una volta arrivato alla conclusione che la mia password non cifrata messa accanto al comando run e successivamente cifrata risulta uguale a quella effettivamente giusta. A questo punto esco dalla modalità debug facendo ctrl-D e mando in esecuzione password con accanto la password non cifrata che ho appena trovato e, se corretta, mi restituisce esito positivo come mostrato in figura.

```
rookie@8d3caaeb9e26:~/Challenge1$ ./password g00dj0b!
Correct password! >>> You logged in! <<<
```